

AI Output Validation

I used the AI mostly to help my understanding of the topics involved in the project, such as the Hindley-Milner type-system or Kennedy's extension; I also used it to explore the codebase of the F# compiler, in order to quickly retrieve the information I needed to check the implementation of the units of measure. In total, I used three different AI tools, each of them for different goals:

- ChatGPT 5.2 ([link to the chat](#)) (with web search always enabled, in order to have references to check the output) was used to help me understand how F# units of measure works, the syntax and other introductory notions about them, including some edge cases and the semantics of the `inline` annotation in F#; I also used it to better understand Hindley-Milner type-system;
- Google's NotebookLM ([link to the chat](#)) was used to have a better understanding of Kennedy's work about integrating units of measure and dimension into a type-system; I decided to use this tool instead of ChatGPT because I directly uploaded the original paper, in order to have references to it in the AI's answers;
- GitHub Copilot Chat ([link to the chat](#)) (with GPT 5.1 Codex-Max and the dotnet/fsharp repository) was used to retrieve information about the implementation of the units of measure in the F# compiler, and I chose to use this tool since it can crawl the entire repository and it can directly reference portions of code in the committed files.

All the AI outputs was validated using trustworthy resources:

- the answers about functioning of the units of measure in F# were checked using the [official specification of the F# language](#);
- the answers about the Hindley-Milner type-system were checked using the [Wikipedia's page for the HM system](#);
- NotebookLM's answers were directly validated using the paper and the references provided in the answers;
- Copilot's answers were validated by reading the related [docs provided in the repository](#), and when possible by reading the comments/documentation in the referenced pieces of code provided by Copilot.

Major errors

During the AI output validation, I noticed some errors in the answers: here are reported the most relevant one, together with the motivation.

- Fractional exponents in unit of measure: when asked if in F# was possible to raise a unit of measure to a fractional exponent, ChatGPT answered no; even the [official language specification](#) reports that exponentiation is supported only with integer exponents, but in the F# interactive interpreter (version 14.0.100.0 for F# 10.0) the output of the command `let x = 1.0<m^(1/2)>` (where `m` is a previously defined unit of measure) is `val x :`

`float<m ^ (1/2)> = 1.0`, meaning that fractional exponents are correctly parsed and maintained; this is also consistent of what I have found by exploring the codebase of the compiler.

- Type casting with measure-annotations: when asked if type-casting function like `float` were measure-polymorphic, ChatGPT answered yes and told that the type of the function `float` is `int<'u> -> float<'u>`; instead, by evaluating just `float` in the interactive interpreter, we discover that its type is just `int -> float`, and if we try to apply it to a measure-annotated value (e.g. `1<m>`), it returns a non-annotated value, consistently with its type; the file `examples/measure-type-cast.fsx` contains a function breaking the extension soundness and another which can be carefully used to cast measure-annotated types.
- Type-polymorphism and measure-polymorphism: when answered about the combination of these two, ChatGPT answered that they were compatible, meaning that it is allowed to write a type of the form `^T<'u>`; by trying to do this, we discover that this is not possible as it is a syntax error.
- Measure-polymorphism of literals: when answered about this, ChatGPT told that, other than zero, also one is a measure-polymorphic literal; as discussed in the report, this can not be the case because it would break soundness, and in fact the type of `1.0<_>` is not `float<'u>` but rather `float`.
- The need of `inline` in the measure-polymorphic square function: after providing an example of square function with the `inline` modifier, I asked it why it was necessary; the given answer reasoned about the need of statically know which implementation of `*` should be used; but since measures are erased at runtime, the correct implementation can be directly inferenced without `inline`, which is in fact confirmed by the type emitted by the interactive interpreter after evaluating the square function `let square (x: float<'u>) = x * x`, which is `float<'u> -> float<'u^2>`.